MODULE-(III)

# TRANSCATION PROCESSING CONCEPTS

→ The Concept of transcation provides a mechanism for describing logical units of database processing. Transcation processing systems are systems with large database and hundreds of concurrent users executing database-transcations.

Example:- Airline reservations, banking, credit card, stock-market and so on. These system require high availability and fast response time for hundreds of concurrent users.

→ A DBMS is single if almost one user at a time, can use the system and it is multiuser if many users can use the system and hence access the database concurrently.

Definition: Transcation is an executing program that forms a logical unit of database processing.

— A transcation includes one (or) more database access-operations - these include insertion, deletion, modification (or) retrieval operations.

— one way of specifying the transcation boundaries is by specifying explicit begin transcation and end-transcation statements in an application program.

* If the database operations in a transcation do not update the database but only retrieve the data, the transcation is called a read only transcations.

→ The basic operations that a transcation can include are as follows:-

① Read-item(X): Reads a database item named in a program variable.

② Write-item(X): Writes the value of program variable X into the database item named X.

— A transaction includes read item and write item operations to access and update the database.

Ex:

$T_2$

```
read item(X)
X := X + M
write item(X)
```

The read-set of a transaction is the set of all items that the transaction reads, and write set is the set of all items that the transaction writes. Here read set of $T_2$ is $\{X, Y\}$ and write set also $\{X, Y\}$.

→ Concurrency control and recovery mechanisms are mainly concerned with the database access commands in a transaction. Transactions submitted by the various users may execute concurrently and may access and update the same database items. If this concurrent execution is uncontrolled, it may lead to problems, such as an inconsistent database.

## WHY CONCURRENCY CONTROL IS NEEDED?

— Several problems can occur when concurrent transactions execute in an uncontrolled manner.

Example: Airline reservations database in which a record is stored for each airline flight. Each record includes the no of reserved seats on that flight as a named data item among other information.

| $T_1$ | $T_2$ |
|---|---|
| read item(X);<br>X = X − N;<br>write item(X);<br>read item(Y);<br>Y = Y + N;<br>write item(Y); | read item(X);<br>X = X + M;<br>write item(X); |

In $T_1$: Here Transaction $T_1$ that transfers N reservations from one flight whose no of reserved seats is stored in the database item named X to another flight whose no of reserved seats is stored in the database item named

In $T_2$: transaction $T_2$ reserves M seats on the first flight(X):

## WHY CONCURRENCY CONTROL IS NEEDED?

Concurrency control and recovery mechanisms are mainly concerned with the database access commands in a transaction.

Transactions submitted by various users may execute concurrently and may access and update the same database items. if this concurrent execution is uncontrolled, it may lead to problems such as an inconsistent database.

Several problems can occur when concurrent transactions execute in an uncontrolled manner.

The concurrency control has the following three main problems:

(i) the lost update problem (write-write conflict)

(ii) the dirty read or temporary update problem (write_read conflict)

(iii) the incorrect summary problem.

example:

Airline reservation database in which a record is stored for each airline flight. Each record includes the number of reserved seats for that flight.

| | T1 | T2 |
|---|---|---|
| 80 | Readitem(X) | |
| 80-5=75 | X:=X-N; | |
| X = 75 | writeitem(X) | |
| | | readitem(X) → 75 |
| | | X:=X+M; → 75+4=79 |
| | | writeitem(X) → 79 |
| 80 | readitem(Y) | |
| Y=80 4= 84 | Y:=Y+N | |
| | writeitem(Y) | |

Y = 84

In T1

In transaction $T_1$ that transfers N reservations from one flight whose number of reserved seats is stored in the dataitem named 'X' to another flight whose number of reserved seats is stored in the dataitem named 'Y'.

In T2

Transaction T2 reserves 'M' number of seats to the first flight whose reserved seats is stored in the data item named 'X'

(1) The lost update problem or (write-write conflict) :

A lost update problem occurs when two transactions that acce the same database items have their operations in a way that makes value of some database item incorrect.

Suppose T1 and T2 are submitted approximately at the sam and suppose their operations are interleaved.

The final value of ITEM(X) is incorrect because T2 reads value of X before T1 changes it in the database and hence the updated value resulting from T1 is lost.

example :

if X = 80 initially, N = 5 (T1 i.e T1 transfers 5 seats reservation from the flight corresponding to 'X' to the flight corresponding to 'Y'. and M = 4 (T2 reserves 4 seats on 'X')

| T1 | | T2 | |
|---|---|---|---|
| X = 80 | readitem(X) | | |
| X = 80 − 5 = 75 | X := X − N | | |
| | | readitem(X) | ⟶ X = 80 |
| | | X := X + M | ⟶ X = 80 + 4 = 84 |
| X = 84 | writeitem(X) | | |
| Y = 80 | readitem(Y) | | |
| | | writeitem(X) | ⟶ 84 |
| Y = 80 + 5 = 85 | Y := Y + N | | |
| Y = 85 | writeitem(Y) | | |

The final result should be X = 79, but in the interleaving of operations it is X = 84, because the update in T1 that removed the five seats from X was lost.

Item 'X' has incorrect value because its updated by T1 is lost or overwritten.

he tempory update

## dirty update problem:

A dirty read problem occurs when one transaction updates a data base item and then the transaction fails for some reason. The updated database item is read or accessed by another transaction before it changed back to the original value.

| T1 | T2 |
|---|---|
| 80 readitem(x) | |
| x=80-5 x:= x-N | |
| =75 writeitem(x) | |
| | readitem(X) 75 |
| | X:= Y+M 75+4 |
| | writeitem(Y) 79 |

Transaction T1 fails and most change the value of x back to its original or old value. mean while T2 has read the temporary incorre value of x.

Due to failure of T1, the value of item x that is read by T2 is called dirty data because it has been created by a transaction that has not completed and committed yet. Hence this problem is also known as dirty read problem.

## ③ The incorrect Summary Problem:

The incorrect summary problem occur if a transaction writes a data object previously read by another transaction.

it means that a transaction reads several values from a database while a second transaction updates some of them.

Suppose a transaction T3 is executing the total number of reservations on all the flights mean while transaction T1 is executing. if the interleaving of operation occurs, the result value of T3 will be off by an amount N,

| T1 | T3 |
|---|---|
| | Sum := 0 |
| | ~~Sum := sum+A~~ |
| | read item (A) |
| | Sum = Sum + A; |
| | . |
| | . |
| | . |
| read item (X) | readitem(X) |
| X := X - N; | Sum := Sum + X |
| write item (X) | read item (Y) |
| | sum = sum + Y; |
| readitem(Y) | |
| Y := Y + N | |
| writeitem (Y) | |

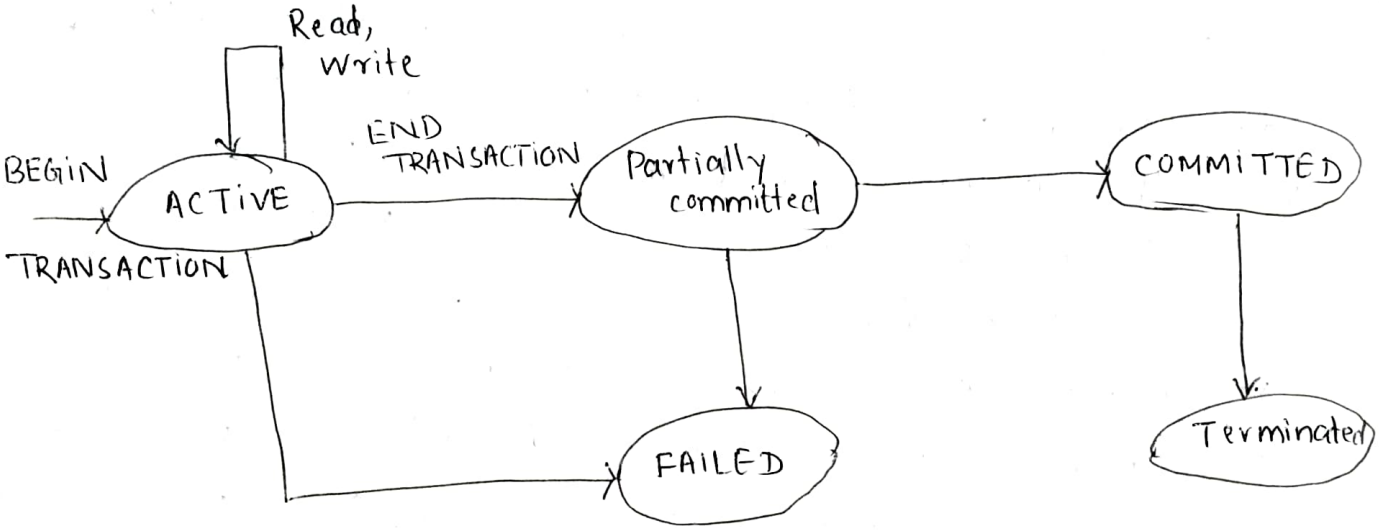
because $T_3$ reads the value of X after N seats have been subtracted from it but read the value of Y before the N seats have been added to it.

$T_3$ reads X after N is subtracted and reads Y before N is added, a wrong summary is the result off by N.

## Transaction States :

A transaction is an atomic unit of work that is either performed completely or not at all performed.

For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits. Therefore, the recovery manager keep track of the following operations.




① **BEGIN Transaction**

This marks the beginning of the transaction execution.

② **READ or WRITE**

These specify read or write operation on the database items that are executed as parts of a transaction.

③ **END Transaction**

These specify the Read and write transaction operations have ended and marks the end of the transaction.

④ **commit transaction**

This signals the successful end of transaction so that any changes executed by the transaction can be

safely committed to the database.

(5) Rollback or ABORT :

This signals that the transaction has ended unsuccessfully so that any changes or effects that the transaction may have applied to the database must be cancelled.

Description

A transaction goes into an active state immediately after it starts execution, where it can issue Read and write operations. when the transaction ends it moves to the partially committed state. At this point some recovery protocols needed to ensure that a system failure will not result in an inability to record the changes of the transaction permanently. Once this check is successful, the transaction is said to have reached its commit point and enters the committed state. Once a transaction is committed, it has concluded its execution successfully and all its changes must be recorded permanently in the database.

How ever the transaction can go to the failed state if one of the check fails if the transaction is terminated during its active state. The transaction may then have to rolled back to undo the effect of its write operations on the database. The terminated state corresponds to the transaction leaving the system.

## Schedule :

When transactions are executing concurrently in an interleaved fashion then the order of execution of operations from the various transaction is known as schedule.

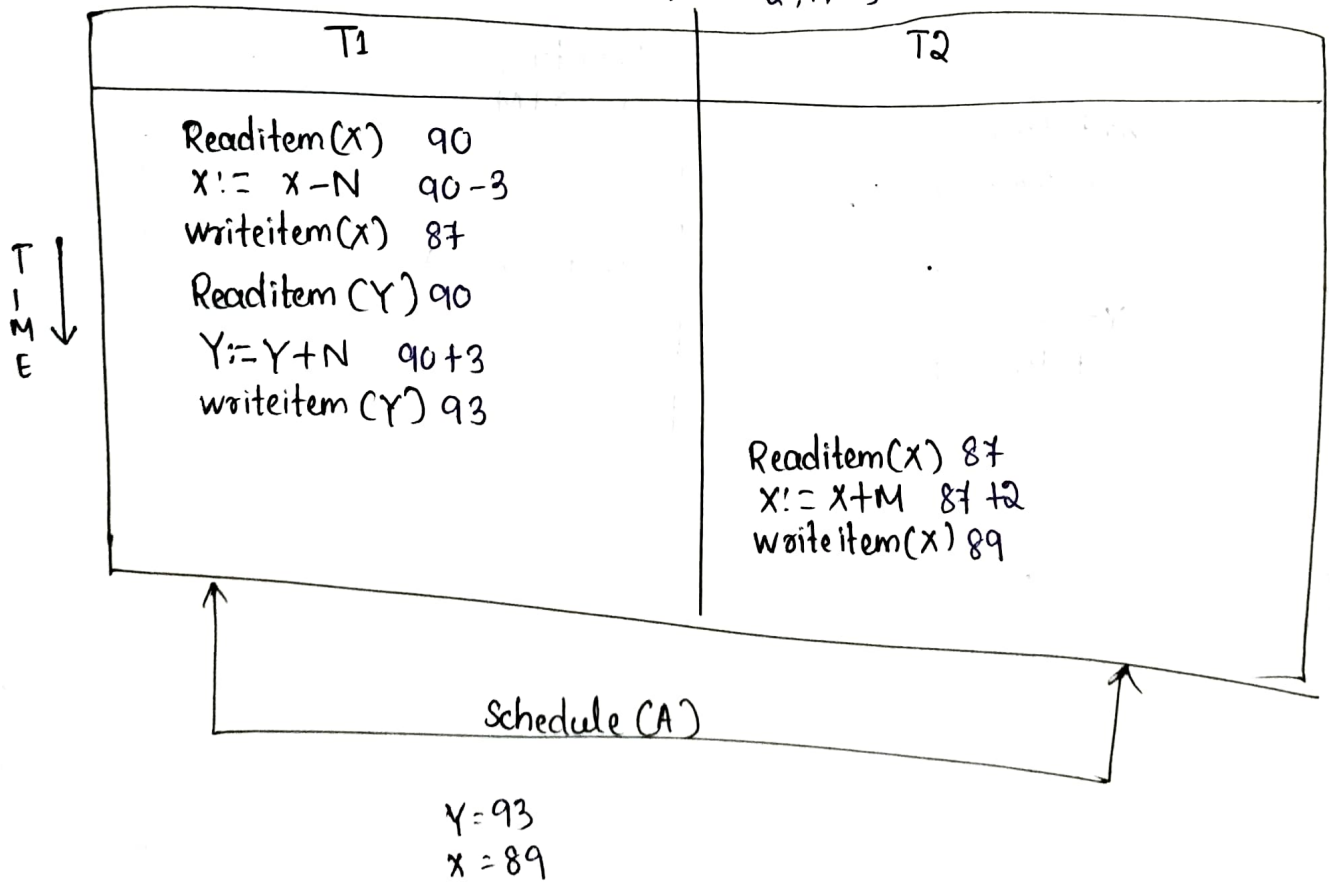
A schedule is a list of actions from a set of transactions, must consist of all instructions of those transactions and must preserve the order in which the instructions appear in each individual transaction.

## Serializability Of Schedules :

→ The concept of serializability of schedules is used to identify which schedules are correct when transaction executions have interleaving of their operations in the schedule.

→ The objective of serializability schedules is to find non-serial schedules that allow transactions to execute concurrently with out interfering with one another.

Let X=90, Y=90, M=2, N=3




Y = 93
X = 89

| T1 | T2 |
|---|---|
| | Readitem(X) 90 |
| | X := X+M 90+2 = 92 |
| | writeitem(X) 92 |
| Readitem(X) 92 | |
| X := X-N 92-3 = 89 | |
| writeitem(X) 89 | |
| Readitem(Y) 90 | |
| Y := Y+N 90+3 = 93 | |
| writeitem(Y) 93 | |

TIME ↓

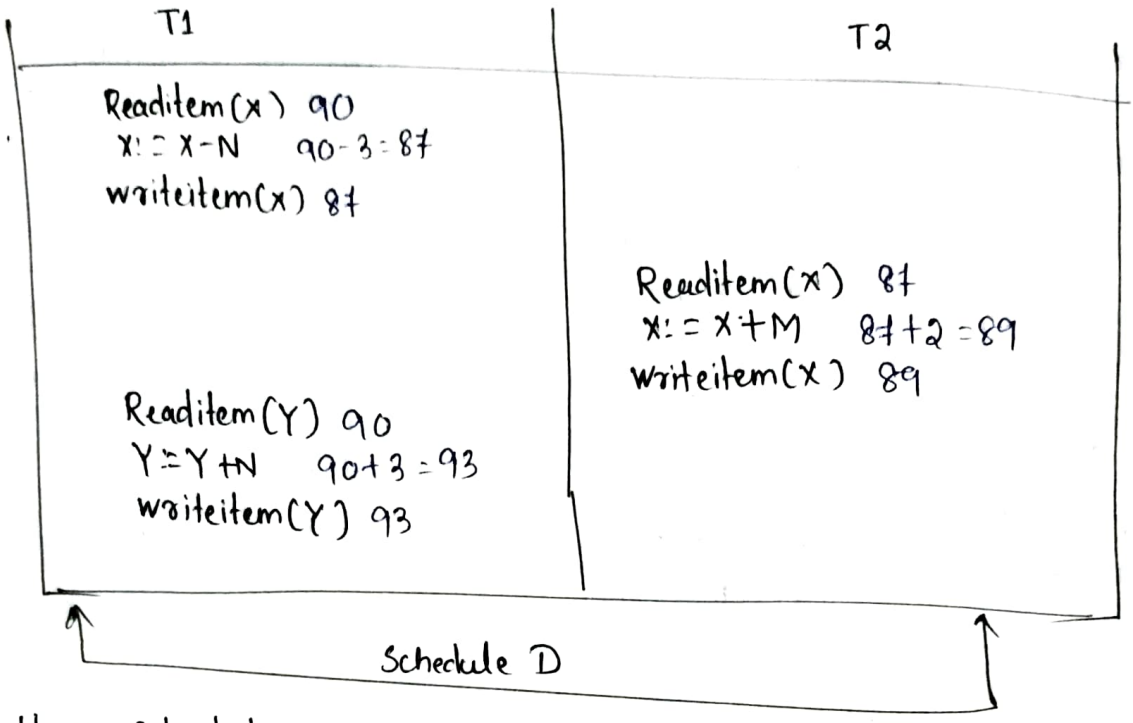
schedule B

X = 89
Y = 93

| T1 | T2 |
|---|---|
| Readitem(X) 90 | |
| X := X-N 90-3 = 87 | |
| | Readitem(X) 90 |
| | X := X+M 90+2 = 92 |
| writeitem(X) | |
| Readitem(Y) | |
| | writeitem(X) |
| Y = Y+N | |
| writeitem(Y) | |

TIME ↓

X = a2
X = a3




→ Here Schedule A and B are called Serial, because the operations of each transaction are executed consequtively, without any interleaved operation from other transaction.

→ In Serial schedule entire transactions are performed in a serial order T1 and then T2 for schedule A and T2 and T1 for schedule B.

→ Schedule C and D are called NonSerial because each sequence interleaves operations from the two transaction.

→ Finally a Schedule S is called Serial if, for every transaction T Participating in the Schedule, all the operations of T are executed consequetively in the schedule; other wise the schedule is nonserial.

→ In Serial t Schedule, if a transaction waits for an I/O operations to complete, we can not switch the CPU processor to another transaction, thus wasting valuable CPU processing time.

→ Suppose the initial values of database items are
X=90, Y=90, M=2, and N=3
Results for schedules are:-

Schedule A (X=89 and Y=93)
schedule B (X=89 and Y=93)
Schedule C (X=92 and Y=93) ← incorrect
Schedule D (X=89 and Y=93

→ Here Some Nonserial schedules always give a correct result may give erroneous result. The concept used to schedules manner is that of serializability of a schedule.

→ A Schedule 'S' of 'n' transactions is serializable if it is equiva to some serial schedule of the same 'n' transactions.

→ we can form two disjoint groups of the nonserial schedules are equivalent to one or more of the serial schedules hence are serializable and those that are not equivalent to Serial schedule and hence are not serializable.

→ In the above four schedules, Schedules A, B and D are Serializable where as schedule C is not serializable.

## Result Equivalence:

Two schedules are called result equivalent if they produce the same final state of the database.

example:

| S1 | S2 |
|---|---|
| Readitem(x)<br>x := x + 10;<br>writeitem(x) | Readitem(x)<br>x := x * 1.1;<br>writeitem(x)<br><br>Read item |

Here Schedule S1 and S2 will produce the same final value of the database state if they execute on database with an initial value of x=100, but for other initial values of x, the schedules are not result equivalence.

Hence result equivalent alone can not be used to define equivalence of schedules. So the most general approach to defining equivalence is to make that for two schedules to be equivalent, the operations applied to each data item affected by the schedules should be applied to that item in both Schedules in the order.

## Currency Control Techniques:

Transaction processing systems usually allow multiple transactions to run concurrently. By allowing multiple transactions to run concurrently will improve the performance of the system in terms of increased throughput or improved response time, but this causes several complications with consistency of the data.

There are some of the main techniques used to control concurrent execution of transactions are:-

① **LOCKING:**

The technique of locking data items to prevent multiple transactions from accessing the data items concurrently.

② **TIME STAMPS:**

A timestamp is a unique identifier for each transaction, generated by the system. Concurrency control protocols that use time stamp ordering to ensure serializability.

③ **MULTIVERSION Concurrency Protocol:**

Which use multiple version of a dataitem.

④ **Validation / Certification:**

Validation of a transaction after it executes its operations, These are some times called as Optimistic protocols.

**LOCKING SCHEME:** A database can be considered as being made upof a set of data items

→ A LOCK is a variable associated with a data item that describes the states of the item with respect to the possible operations that can be applied to it. Manipulating the value of a lock is called LOCKING.

→ The value of a lock variable is used in the locking scheme to control the concurrent access and manipulation of the associated data items.

→ Locking the items being used by a transaction can prevent other concurrently running transactions from using these locked items.

→ The locking is done by a subsystem of the database management system usually called the lock manager.

→ Two types of locks are defined:
① EXCLUSIVE LOCK
② SHARED LOCK.

① EXCLUSIVE LOCK

It is also called an update/ a write lock. The intention of this mode of Locking is to provide exclusive use of the dataitem to one transaction.

If a transaction 'T' locks a dataitem Q in an exclusive mode, no other transactions can access Q, not even to read Q, until the lock is released by transaction T.

② SHARED LOCK:

It is also called a read lock. The intention of this mode of locking is to ensure that the data item does not undergo any modifications while it is locked in this mode.
Any number of transactions can concurrently lock and access a dataitem in the shared mode, but none of these transactions can modify the data item.

→ A data item locked in a shared mode can not be locked in the exclusive mode until the shared lock is released by all transactions holding the lock.

→ A data item locked in the exclusive mode can not be locked in the shared mode until the exclusive lock on the data item is released.

There are three locking operations:

① read lock(x)
② write lock(x)
③ unlock(x)

→ A lock is associated with an item X, LOC(X) now has three possible states.

① read locked
② write locked
③ unlocked.

→ When we use the shared / exclusive locking scheme the system must ensure the following rules: –

① A transaction must issue the operation read lock(x) or write lock(x) before any read_item(x) operation is performed on T.

② A transaction 'T' must issue the operation write lock(x) before any write item(x) operation is performed on T.

③ A transaction 'T' must issue the operation unlock(x) after all readitem(x) and write item(x) operations are completed in T.

④ A transaction 'T' will not issue a read_lock(x) operations if it already holds a read (shared) lock or a write lock on item X.

⑤ A transaction 'T' will not issue a write lock(x) operation if it already holds a read or a write lock on item X.

⑥ A transaction T will not issue a unlock(x) operation unless it already holds a read or write lock on item X.

1103904955 4

## TIME STAMP BASED ORDER :→

→ Time stamp is a unique identifier created by the DBMS to identify a transaction.

→ In the timestamp based method a serial order is created among the concurrent transaction, by assigning to each transaction a unique non decreasing ~~values~~ number.

The usual value assigned to each transaction is the system clock value at the start of transaction, hence the name time stamp ordering.

→ A transaction with a smaller timestamp value is considered ~~as~~ to be an older transaction then ~~other~~ another transaction with a larger time stamp value

→ The serializability that the system enforce is the order of the time stamps of the concurrent transactions.

if two transactions Ti and Tj with time stamp values ti and tj respectively such that ti < tj are to run concurrently; then the schedule produced by the system is equivalent to running older transaction Tc first followed by the ~~o~~ younger one Tj

### Time Stamp Ordering algorithm:

The algorithm with each data base item X two time stamp (Ts) values.

① read Ts(X) :

The read time stamp of item X; this is the largest time stamp among all the timestamps of transactions that have successfully read item(X)

That is

read $Ts(x) = Ts(T)$

where T is the youngest transaction that has read x successfully.

② write Ts(x):

The write time stamp of item(x); this is the largest of all the time stamps of transaction that successfully written item(x).

i.e

write $Ts(x) = Ts(T)$

where T is the youngest transaction that has written x successfully.

→ When ever some transaction T tries to issue a read item(x) or a write item(x) operation, the basic time stamp ordering algorithm To compares the time stamp T with read Ts(x) and write Ts(x) to ensure that the time stamp order of transaction is not violated. if this order is violated, then transaction T is aborted and resubmitted to the system as a new transaction with a new time stamp.

## DEAD LOCKS

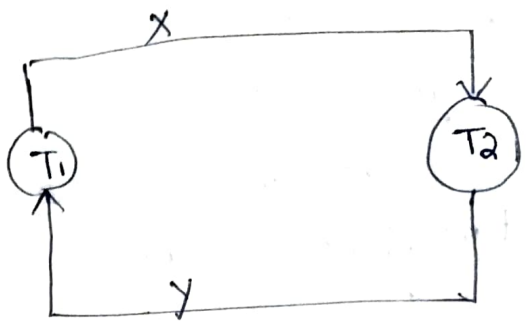
— Deadlock is a specific case of concurrency problem. A concurrency Problem can be resolved and yet ~~it canceled~~ it can lead to deadlock.

— Deadlock occurs when each transaction T in a set of two or more transaction is waiting for some time that is locked by some other transaction T' in the set.

Hence each transaction in the set is on a waiting queue, waiting for one of the other transactions in the set to release the lock on an item.

example :

| $T_1$ | $T_2$ |
|---|---|
| readlock(Y); | |
| readitem(y); | |
| | readlock(x); |
| | readitem(x); |
| writelock(x) | |
| | writelock(Y); |




(wait for graph for a partial schedule)

[A partial schedule of $T_1$ and $T_2$ that is in a state of deadlock]

— In the above two transactions $T_1$ and $T_2$ are deadlocked in a partial schedule; $T_1$ is on the waiting queue for x, which is locked by $T_2$, while $T_2$ is on the waiting queue for y, which is locked by $T_1$. Meanwhile, neither $T_1$ nor $T_2$ nore any other transaction can access items x and y.

— There are two principal methods for dealing with deadlock Problem.

① we can use a deadlock prevention protocal to ensure that the system will never enter a deadlock state.

② we can allow the system to enter a deadlock state and then try to recover by using deadlock detection and deadlock recovery scheme.

## Database Recovery:

→ Recovery means restoration of database to the most recent correct state, if a failure occurs. for this the system keeps records of the changes by transactions in the database in a log.

An ideal strategy for recovery is the following: —

(a) if there occurs a wide damage to the database, due to disk crash or fire, the recovery method restores a past copy of the database from the archive storage. All the committed transactions in the log files are then redone on this restored database.

(b) if the database is not physically damaged but has become erratic due to some logical errors or system crash, the recovery method would undo the operations that caused the error. In this case data in archive storage is not needed, only the logfile is required for undoing.

## Types Of Database Recovery:

Following are some DBMS recovery facilities.

① **checkpoint facility:**
Which enables updates to database be made permanent.

② **Recovery Manager module of the DBMS:**
it is the role of the recovery manager module, available in DBMS package, to guarantee durability and atomicity in case of unpredictable failures.

## Recovery Techniques:

① **Recovery using backups:**

→ it is particularly useful if there has been extensive damage to database

→ Backup are of two types:
(1) Immediate backup
(2) Archival back up.

① Immediate backup:

Which are kept in a floppy disk, hard disk or magnetic tapes when there is system failure, disk crashes, network failure occurs. damage due to virus attack are also repaired using the immediate backup.

② Archival backup:

Which are kept in mass storage devices such as magnetic tape, CDROM, internet services etc. They are useful for recovering data after a earth quake, flood, terrorist attack etc. Archival backup at a seperate place remains safe from thefts, and intentional destruction by user staff.

② Mirroring:

Two complete copies of the database are maintained online on different stable storage devices. This method is mostly used in environments that require nonstop, fault tolerant operations.

③ Recovery Using Transaction log:

Incase of recovery using transaction logs, the following steps are followed.

① The log is search for all the transactions that have recorded a [start transaction] entry, but have not recorded a corresponding [commit] entry.

② These transactions are rolled back.

③ Transactions which have recorded a [commit] entry in the log must have recorded the changes, they did to the database, in the log. These changes are followed to undo their effects on the database.

④ ~~Shadow~~

④ Shadow Paging:

In a single user environment shadow paging system can be used for data, recovery instead of using transaction logs.

## Recovery Procedures:

Recovery from any type of system failure requires the following steps to be performed.

(1) Determining which dataitems are intact and which are would need recovery.

(2) follow the appropriate recovery methods.

(3) Restarting the database so that it can resume normal operations.

(4) Ensure that no dataitems are lost and no incorrect dataitems are entered in the database.

## Deadlock Prevention :→

We can use a deadlock prevention protocol to ensure that the system will never enter into a deadlock state.

We can prevent deadlocks by giving each transaction a priority and ensuring that lower priority transactions are not allowed to wait for higher priority transactions.

One way to assign priorities is to give each transaction a timestamp when it starts up. The lower the timestamp, the higher the transaction's Priority that is the oldest transaction has the highest priority.

If a transaction $T_i$ requests a lock and Transaction $T_j$ holds a conflicting lock the lock manager can use one of the following two Policies.

### ① wait-die :

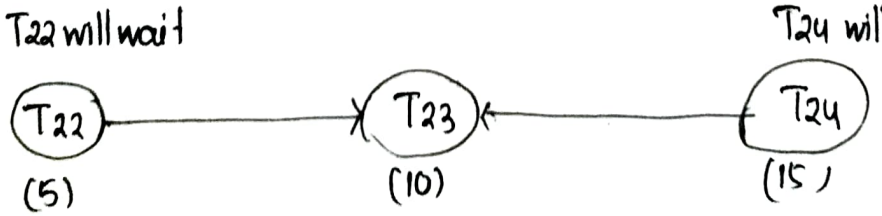
If $T_i$ has higher priority, it is allowed to wait; other wise it is aborted.

it means when transaction $T_i$ requests a dataitem currently held by $T_j$, $T_i$ is allowed to wait only if it has a timestamp smaller than that of $T_j$ (that is $T_i$ is olderthan $T_j$) other wise $T_i$ is rolled back.

example :

Suppose transactions $T_{22}$, $T_{23}$ and $T_{24}$ have timestamps 5, 10 and 15 respectively.

if $T_{22}$ requests a data item held by $T_{23}$ then $T_{22}$ will wait. if $T_{24}$ requests a dataitem held by $T_{23}$ then $T_{24}$ will be rolled back




In the wait-die scheme, lower priority transactions can never wait for higher priority transactions.

(ii) wound wait

An opposite approach to the wait-die scheme is called the wound wait scheme

if $T_i$ has higher priority, abort $T_j$ otherwise $T_i$ waits.

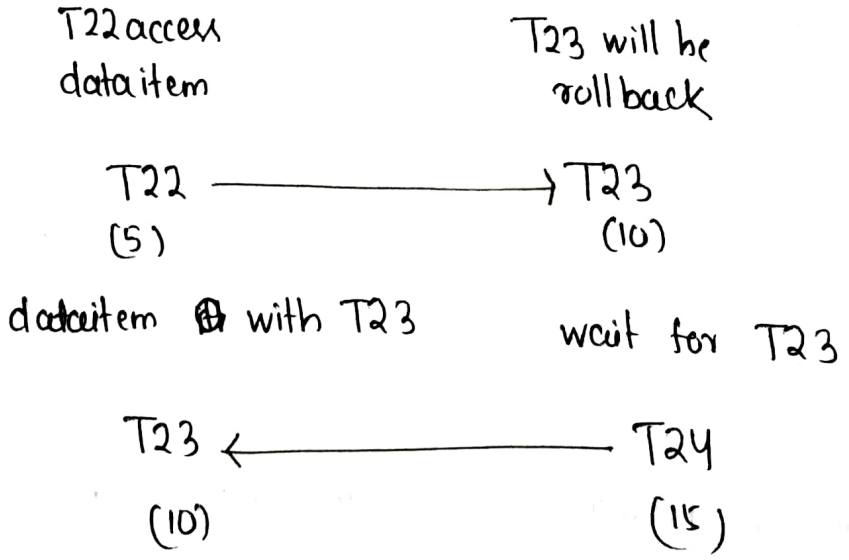
it means when a transaction $T_i$ requests a dataitem currently held by $T_j$, $T_i$ is allowed to wait only if it has a timestamp larger than that of $T_j$ (That is $T_i$ is younger than $T_j$)

otherwise $T_j$ is rolled back ($T_j$ is wounded by $T_i$)

example :

if $T_{22}$ requests a dataitem held by $T_{23}$ then the dataitem will be preempted from $T_{23}$ and $T_{23}$ will be rolled back.

if $T_{24}$ requests a dataitem held by $T_{23}$ then $T_{24}$ will wait




In the wound wait scheme, higher priority transactions never wait for lower priority transactions.

(2) if a younger transaction requests a data item held by an older transaction, the younger transaction is allowed to wait.

## Deadlock detection:

In this the System checks if a state of deadlock actually exists.

(1) A simple way to detect a state of deadlock is for the System to construct and maintain a wait for graph.

(2) Another simple scheme used is timeout. In this if a transaction waits for a period larger than a system defined timeout period, the System assumes that the transaction may be deadlocked and aborts it regardless of whether a deadlock exists or not.